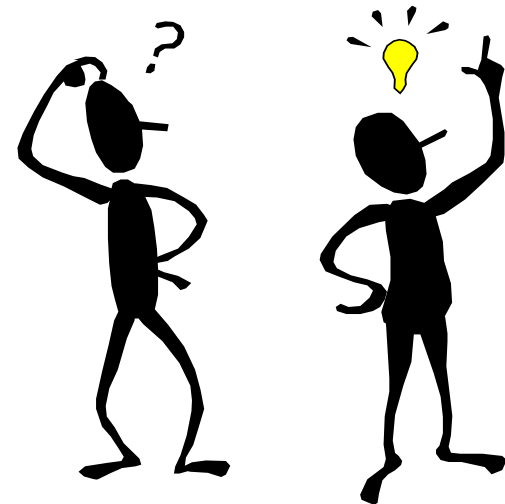


Chapter 3 Text and Image Compression

Contents

- 3.1 Introduction
- 3.2 Compression Principles
- 3.3 Text Compression
 - Huffman coding
 - Arithmetic coding
 - Lempel-Ziv/LZW coding
- 3.4 Images Compression
 - GIF/TIFF/run-length coding
 - JPEG



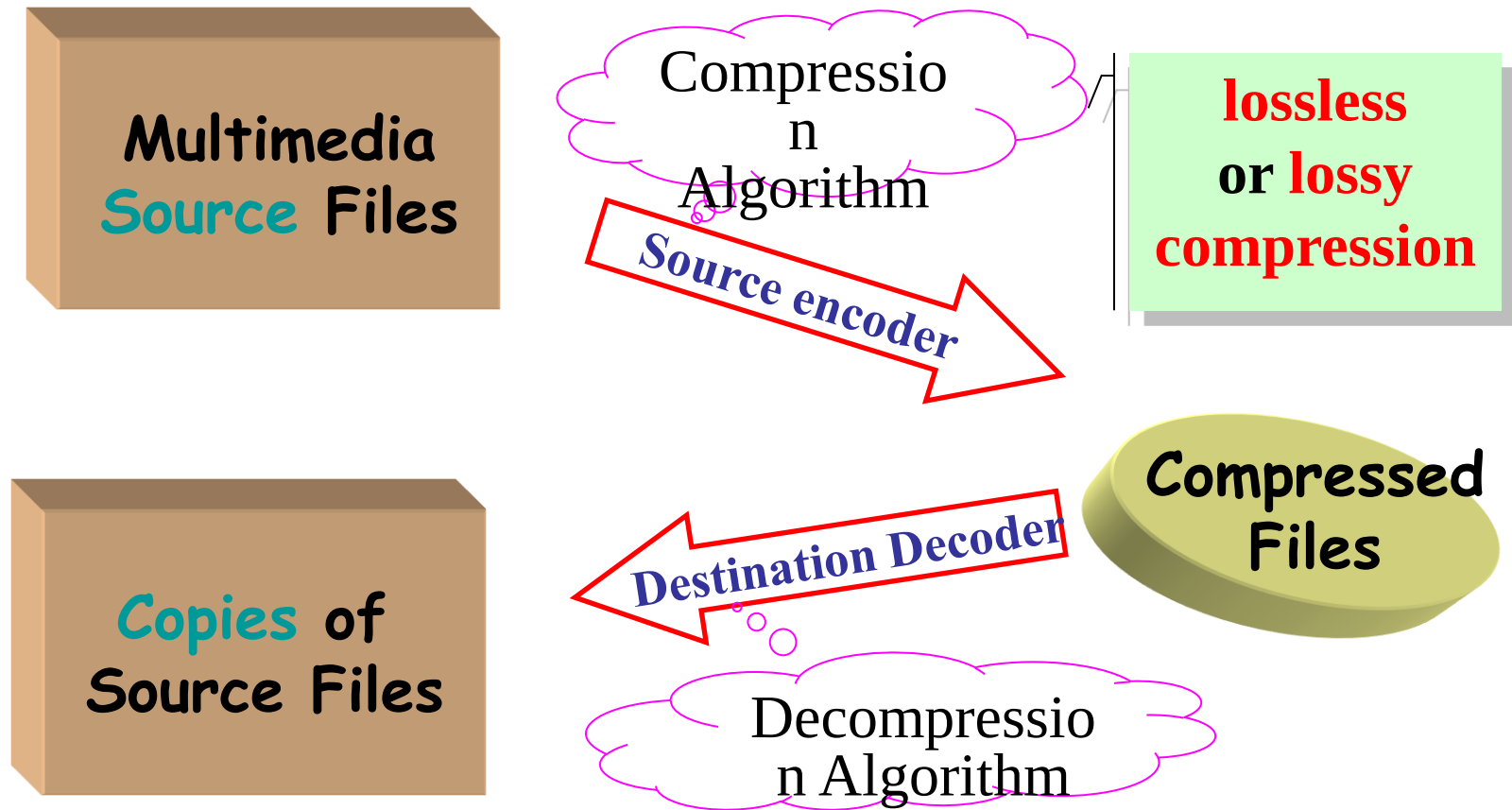
3.1 Introduction

- **Compression** is used to reduce the volume of information to be stored into storages or to reduce the communication bandwidth required for its transmission over the networks



How to put an Elephant into your freezer ? ... !

3.2 Compression Principles



3.2 Compression Principles(2)

RUN: repetitiveness of data

- **Entropy Encoding**

- **Run-length encoding**

- Lossless & Independent of the type of source information
 - Used when the source information comprises long substrings of the same character or binary digit (string or bit pattern, # of occurrences), as FAX e.g) 000000001111111110000011.....

⇒ 0,7 1, 10, 0,5 1,2..... ⇒ 7,10,5,2.....

- **Statistical encoding**

- Based on the probability of occurrence of a pattern
 - The more probable, the shorter codeword
 - "Prefix property": a shorter codeword must not form the start of a longer codeword

3.2 Compression Principles(3)

■ Huffman Encoding

- **Entropy, H** : theoretical min. avg. # of bits that are required to transmit a particular stream

$$H = -\sum_{i=1}^n P_i \log_2 P_i$$

where n : # of symbols, P_i : probability of symbol i

- **Efficiency, E** = H/H'

where, H' = avr. # of bits per codeword = $\sum_{i=1}^n N_i P_i$

N_i : # of bits of symbol i

- E.g) symbols M(10), F(11), Y(010), N(011), O(000), I(001) with probabilities 0.25, 0.25, 0.125, 0.125, 0.125, 0.125
 - $H' = \sum_{i=1}^6 N_i P_i = (2(2 \times 0.25) + 4(3 \times 0.125)) = 2.5$ bits/codeword
 - $H = -\sum_{i=1}^6 P_i \log_2 P_i = -(2(0.25 \log_2 0.25) + 4(0.125 \log_2 0.125)) = 2.5$
 - $E = H/H' = 100\%$
 - 3-bit/codeword if we use fixed-length codewords for six symbols

3.2 Compression Principles(4)

- Source Encoding
 - Differential encoding
 - Small codewords are used each of which indicates only the **difference** in amplitude between the current value/signal being encoded and the immediately preceding value/signal
 - Delta PCM and ADPCM for Audio
 - Transform encoding
 - Transforming the source information from one form into another which is more readily compressible
 - **Spatial Frequency**: changes in (x,y) space
 - **Eyes are more sensitive to the lower frequency** than higher
 - JPEG for Image (DCT-Discrete Cosine Transform)

Not too many changes occur within a few pixels.

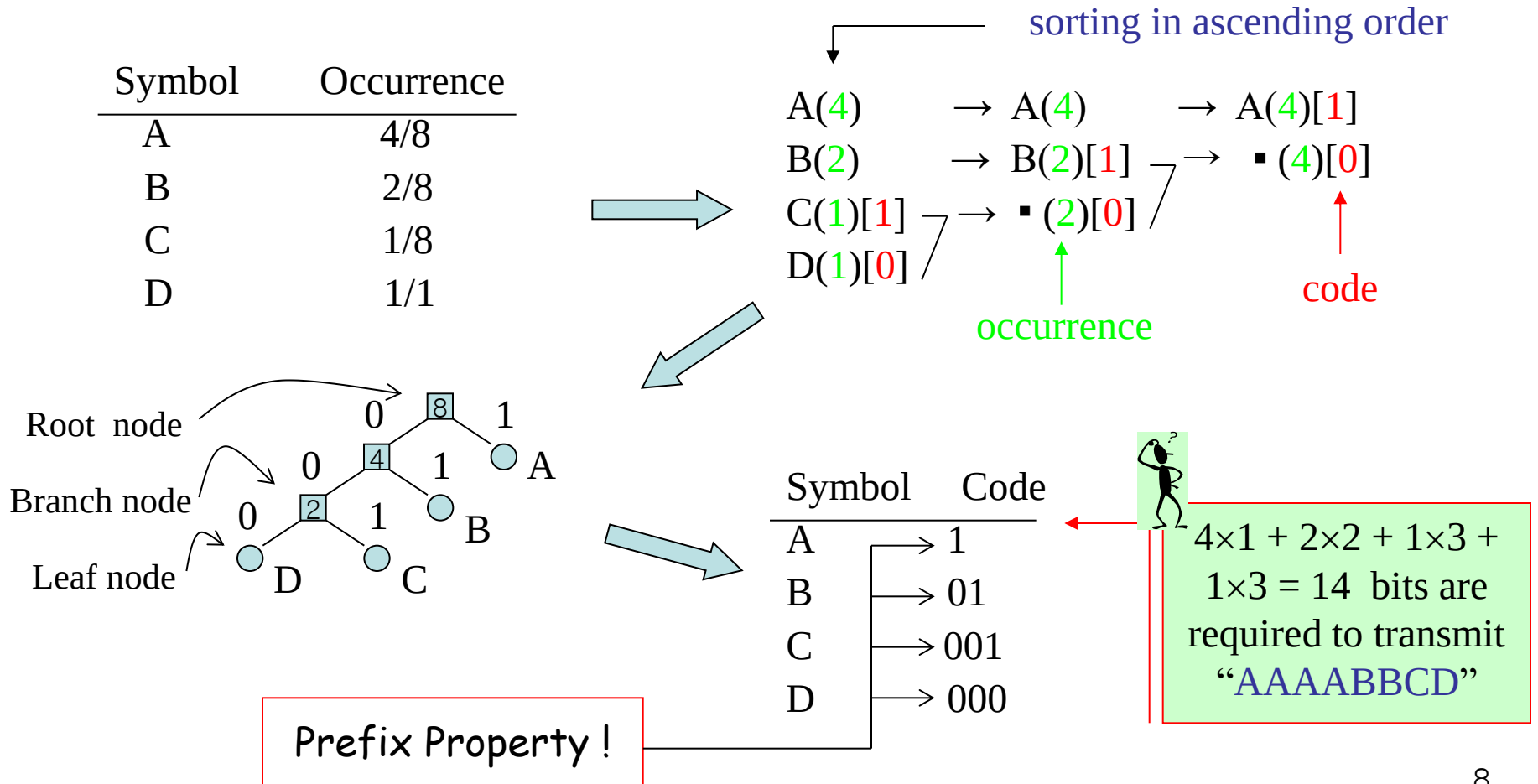
3.3 Text Compression

- Text must be **lossless** 'cause loss of some characters may change the meaning
 - Character-based frequency counting
 - **Huffman Encoding, Arithmetic Encoding**
 - Word-based frequency counting
 - **Lempel-Ziv-Welch (LZW) algorithm**
- **Static coding**: optimum set of variable-length codewords is derived, provided that **relative frequencies of character occurrence is given in priori**
- **Dynamic or Adaptive Coding**: the codewords for a source information are **derived as the transfer of it takes place**. This is done by building up knowledge of both the characters that are present in the text and their relative frequency of occurrence dynamically as the characters are being transmitted

Static Huffman Coding

■ Huffman (Code) Tree

- Given : a number of symbols (or characters) and their relative probabilities *in prior*
- Must hold “*prefix property*” among codes

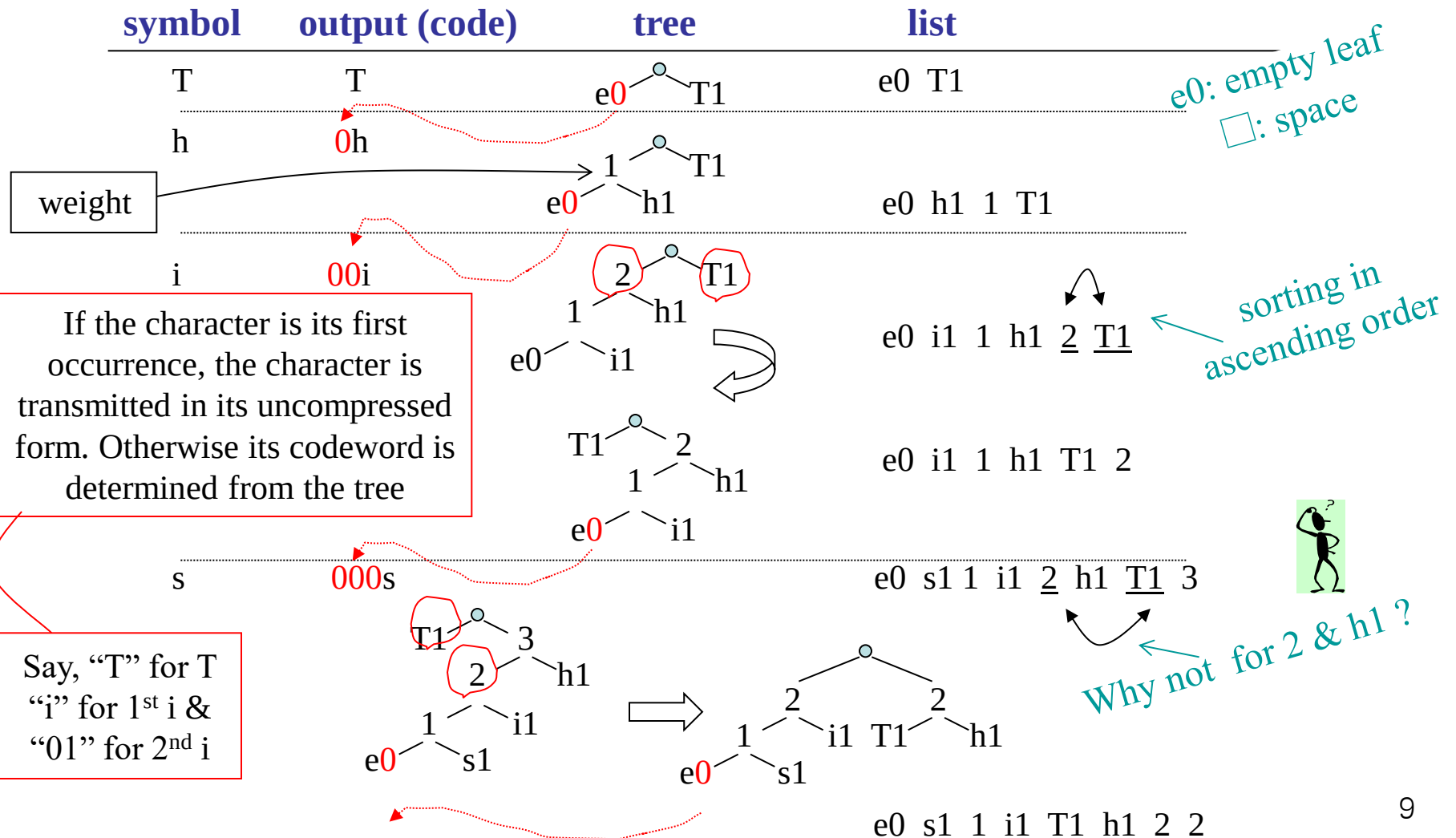


Dynamic Huffman Coding(1)

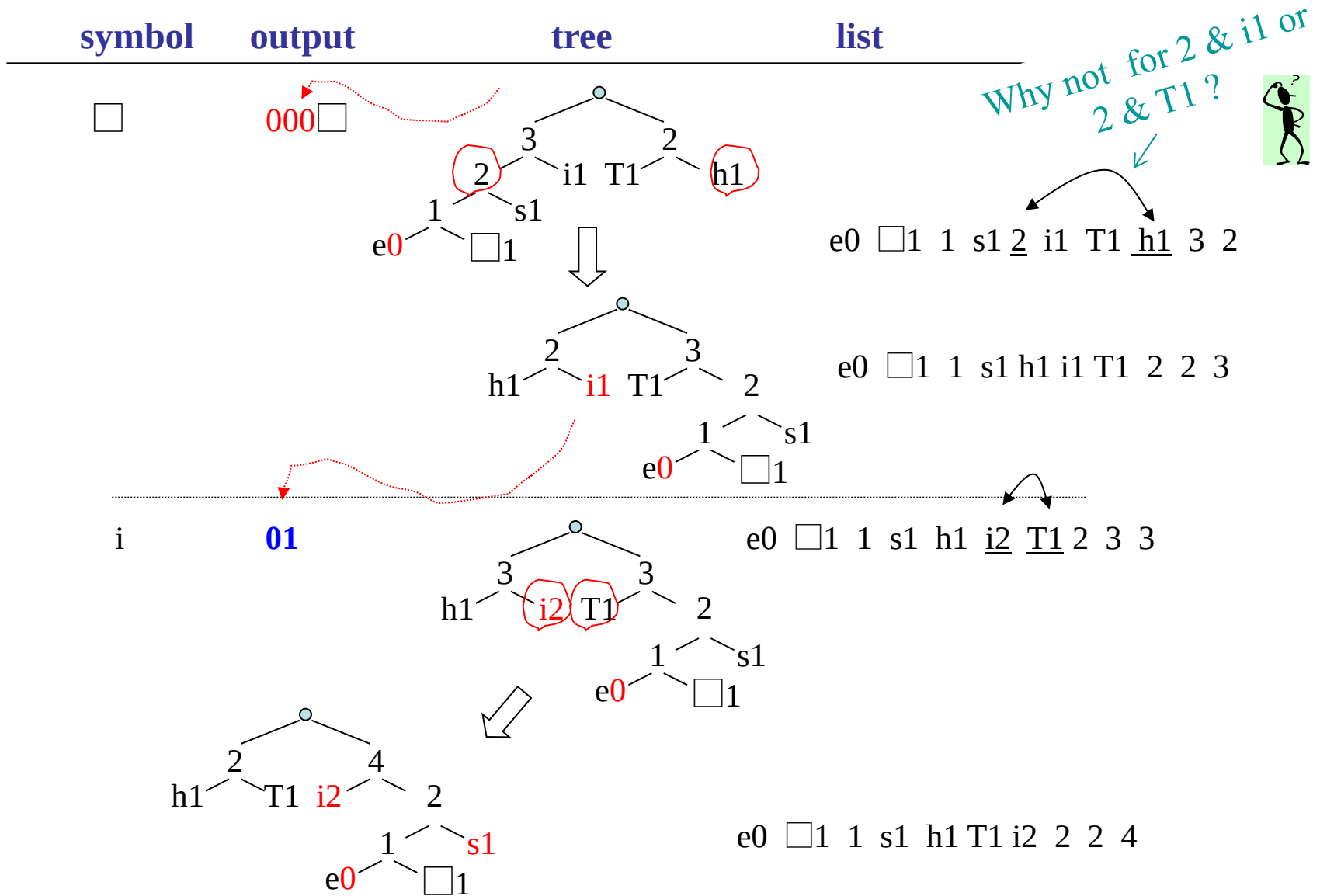
- Huffman (Code) Tree is built **dynamically** as the characters are being transmitted/received
- "**This** **is**" is encoded/decoded as it follows



Initial tree



Dynamic Huffman Coding(2)

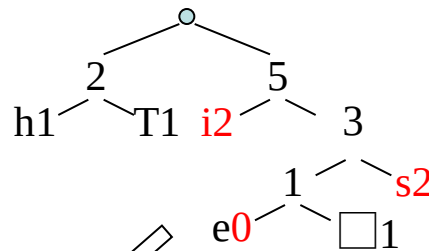


Dynamic Huffman Coding(3)

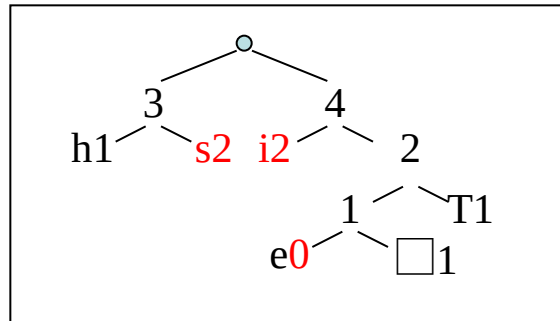
symbol	output	tree	list
--------	--------	------	------

s

111



Why not for s2 & h1?



e0 □ 1 1 s2 h1 T1 i2 3 2 5

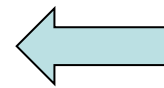
e0 □ 1 1 T1 h1 s2 i2 2 3 4

The compression result : This□01111

If the next character is



T	⇒	111
h	⇒	00
i	⇒	10
s	⇒	01
□	⇒	1101
Other X	⇒	X



Repeat
“Sort the weights & Reconstruct the Tree”
until end of a source file

Arithmetic Coding (2)

Given characters & their probabilities

e=0.3 n=0.3 t=0.2 w=0.1 .=0.1(in alphabet order)

Encode the word “went.”.

Symbol	low	high	range
	0	1.0	1.0
w	0.8	0.9	0.1
e	0.8	0.83	0.03
n	0.809	0.818	0.009
t	0.8144	0.8162	0.0018
.	0.81602	0.8162	0.00018

$$0.1 * 0.3 * 0.3 * 0.2 * 0.1$$

$$0 + 1.0 * 0.8 = 0.8$$

$$0.8 + 0.1 * 0 = 0.8$$

$$0.8 + 0.03 * 0.3 = 0.809$$

$$0.809 + 0.009 * 0.6 = 0.8144$$

$$0.8144 + 0.0018 * 0.9 = 0.81602$$

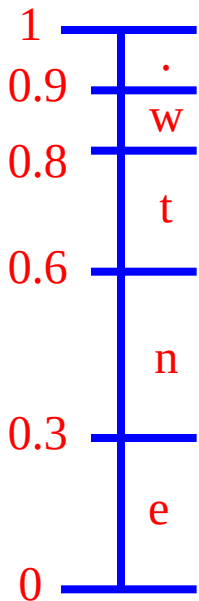
$$0 + 1.0 * 0.9 = 0.9$$

$$0.8 + 0.1 * 0.3 = 0.83$$

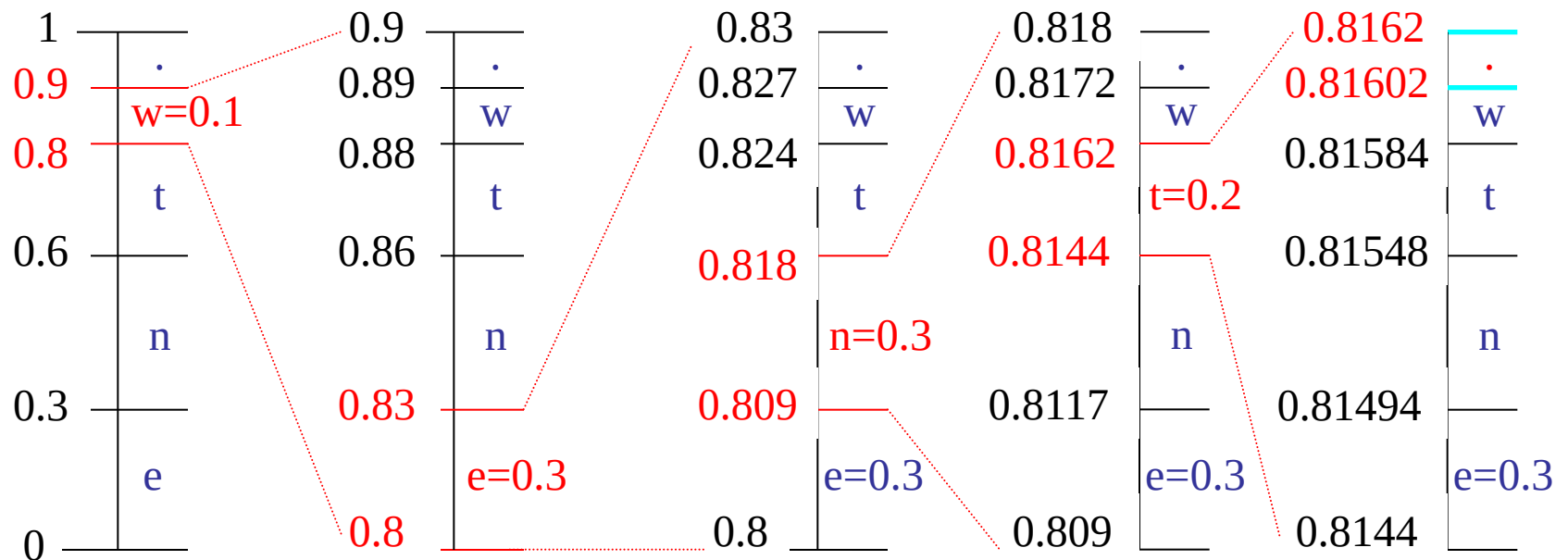
$$0.8 + 0.03 * 0.6 = 0.818$$

$$0.809 + 0.009 * 0.8 = 0.8162$$

$$0.8144 + 0.0018 * 1 = 0.8162$$



Arithmetic Coding (3)



Arithmetic Coding (4)

As low=0.81602, high=0.8162, the codeword for the “went.” is given as follows:

$(0.1)_{10}=0.5$ and $0.5 < \text{high}$	$\rightarrow 0.1$	1st
$(0.01)_{10}=0.25$ and $0.5+0.25(=0.8) < \text{high}$	$\rightarrow 0.01$	2nd
$(0.001)_{10}=0.125$ and $0.8+0.125(=0.925) > \text{high}$	$\rightarrow 0.000$	
.....		
$(0.000001)_{10}=0.015625$ and $0.8+0.015625(=0.815625) < \text{high}$	$\rightarrow 0.000001$	6th
$(0.0000001)_{10}=0.0078125$ and $0.815625+0.015625(=0.8234375) > \text{high}$	$\rightarrow 0.0000000$	
.....		
$(0.0000000000001)_{10}=0.00024406$ and $0.815625+0.00024406 (=0.81586906) < \text{high}$	$\rightarrow 0.0000000000001$	12th
$(0.00000000000001)_{10}=0.00012203$ and $0.81586906+0.00012203 (=0.81599163) < \text{high}$	$\rightarrow 0.00000000000001$	13th
$(0.000000000000001)_{10}=0.000061015$ and $0.81599163+0.000061015 (=0.81605264) < \text{high}$	$\rightarrow 0.000000000000001$	14th

We now have the code “11000100000111” that denotes the bit string “0.11000100000111” (=0.81605264). $\rightarrow \text{cr} = [7\text{-bit} * 5 \text{ symbols}] / [14\text{-bit}] = 2.5$

Lempel-Ziv-Welch(LZW) Coding

- Adaptive (word) dictionary-based compression algorithm
- Send only the index of where the word is stored in the dictionary as each word in a source file encounters
 - Say, a 15-bit suffices for 25,000 words in a typical word-processor
 - A 15-bit index (codeword) for "multimedia" which is represented by 70-bit ASCII codes, and this results in 4.7:1 compression ratio
- A copy of the dictionary must be held by both the sender and the receiver before the coding/decoding. Hence, the dictionary must be built up dynamically as the compressed text is being transmitted

Assume 1) the average number of characters per word is 6, and 2) the dictionary used contains 4096(2^{12}) words. Find the average compression ratio that is achieved relative to using 7-bit ASCII codewords.

The index of the dictionary is given by 12 bits since $4096 = 2^{12}$. A word of average 6 characters is represented by $6 \times 7 (=42)$ bits using ASCII codewords. It follows that $42/12 = 3.5:1$ (350% compression ratio, cr)

Lempel-Ziv-Welch Coding(1)

- A **dynamic** version of a (word) Dictionary-based compression algorithm
 - **Initially**, the dictionary held by both the encoder and decoder contains **only the character set**, say, **ASCII code table** that has been used to create the text
 - **The remaining entries in the dictionary are built up dynamically by both the encoder and decoder** and contains the words that occur in the text
 - For instance, if the character set comprises 128 characters and the dictionary is limited to $4096(2^{12})$ entries.
 - The first 128 entries of the dictionary contain the 128 single characters
 - The remaining $3968(=4096-128)$ entries would contain various words that occur in the source
- **The more frequently** the word stored in the dictionary, **the higher** the level of compression

Lempel-Ziv-Welch Coding(3)

1. Assume, initially, we have a very simple dictionary, i.e., string table

2. We are going to compress the string "ABABBABCABABBA"

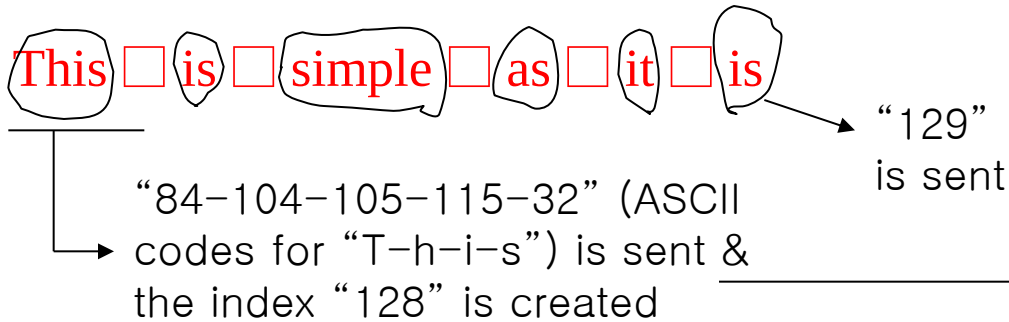
Code	string
1	A
2	B
3	C

s	c	output	code	string
A	B	1	4	AB
B	A	2	5	BA
A	B			
AB	B	4	6	ABB
B	A			
BA	B	5	7	BAB
B	C	2	8	BC
C	A	3	9	CA
A	B			
AB	A	4	10	ABA
A	B			
AB	B			
ABB	A	6	11	ABBA
A	EOF	1		

The output is
124523461 and
 $cr = 14/9 = 1.56$

Lempel-Ziv-Welch Coding(4)

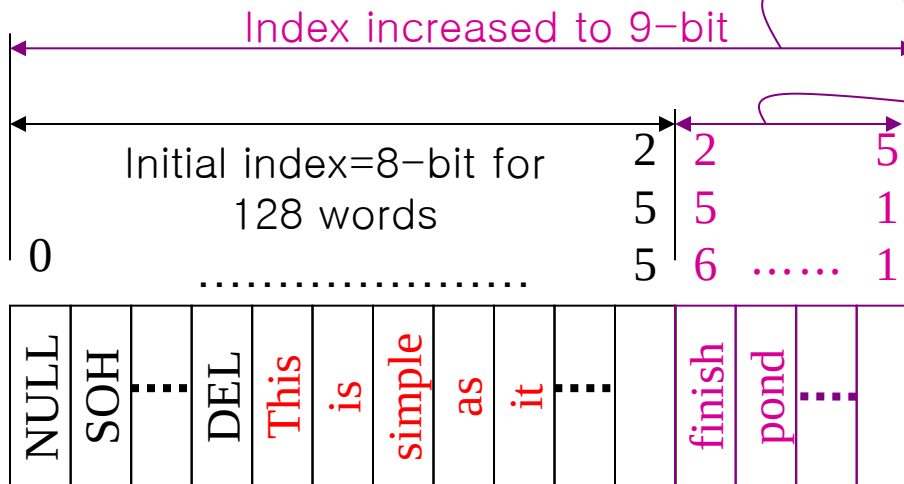
Dictionary contents
(index=8-bit)



NULL	0
SOH	1
⋮	
DEL	127
This	128
is	129
simple	130
as	131
it	132
⋮	
	255

Basic Character Set

Words That Appear First



When the entries becomes insufficient, another 128 entries are created (i.e., double the size of the dictionary)



Lempel-Ziv-Welch Coding(5)

A typical LZW implementation for textual data uses a 12-bit codelength: its dictionary can contain up to 4,096 entries, with the first 256(0-255) entries being ASCII codes using 8-bit.

```
s = NIL;  
while s != end-of-file  
{
```

Decoding Algorithm

```
    k = next input code;  
    entry = dictionary entry for k;  
  
    if (entry == NULL) // exception handling for decoding  
        entry = s+s[0]; // the anomaly case such as ch+st+ch
```

```
    output entry; // a word match: restored (decoded) !
```

```
    if (s != NIL)  
        add s+entry[0] to dictionary with a new code;  
    s = entry;
```

```
}
```

Lempel-Ziv-Welch Coding(6)

Let's decode for the string "ABABBABCABABBA"

Code	string
1	A
2	B
3	C

s	k	entry/output	code	string
NIL	1	A		
A	2	B	4	AB
B	4	AB	5	BA
AB	5	BA	6	ABB
BA	2	B	7	BAB
B	3	C	8	BC
C	4	AB	9	CA
AB	6	ABB	10	ABA
ABB	1	A	11	ABBA
A	EOF			

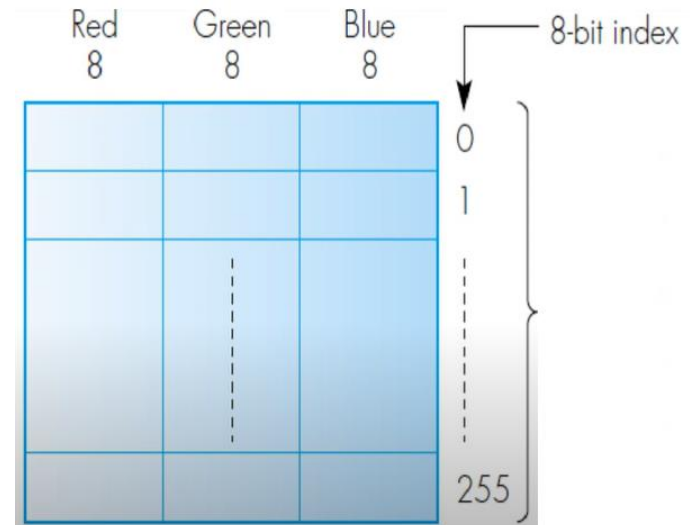
The output is ABABBABCABABBA.

3.4 Image Compression

- Images
 - Computer-generated images say, GIF or TIFF files
 - Digitized images say, such as FAX or MPEG files
- Basically, images are represented (displayed) in 2-d matrix of pixels but generated ones are stored differently in various file systems

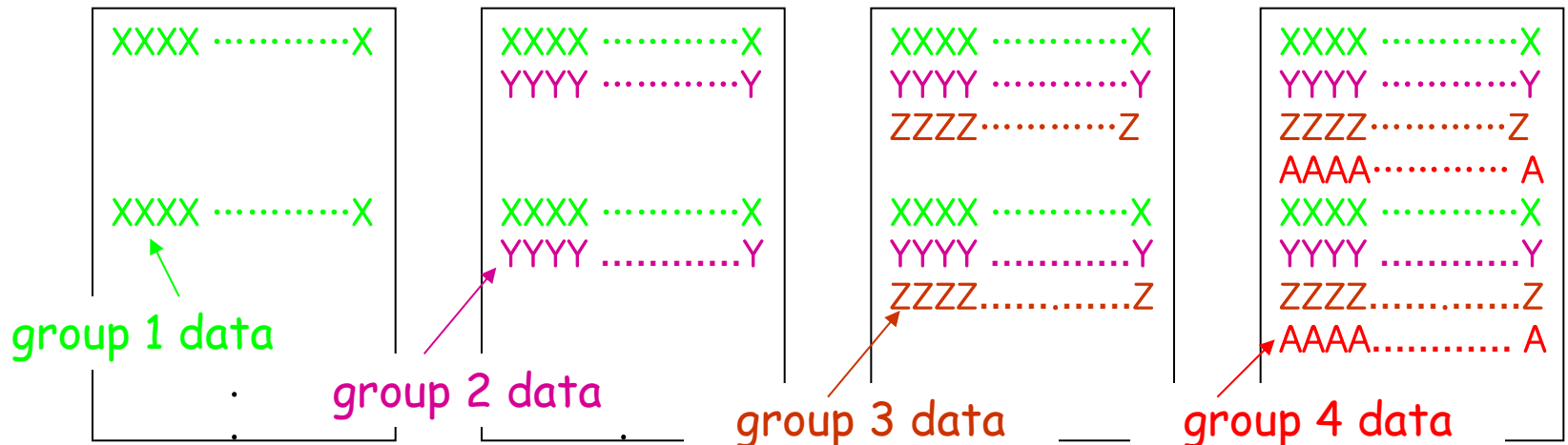
Graphics Interchange Format (GIF)

- Widely used in Internet environments
 - Developed by UNISYS and CompuServe
 - 24-bit pixels are supported: 8-bit for each R, G & B
 - Only 256 colors out of original 2^{24} colors are chosen which match most closely those used in the source
 - Instead of sending each pixel as a 24-bit value, only the 8-bit index to the color table entry that contains the closest match color to the original is sent $\Rightarrow$ 3:1 compression ratio



Graphics Interchange Format (2)

- The **contents of the color table** are sent across the network together with the **compressed image data** and other information such as the **screen size** and **aspect ratio** where, the color table is *either*
 - **Global color table** relates to the whole image to be sent *or*
 - **Local color table** relates to the portion of the whole image
- GIF also allows an **image** to be stored and subsequently transferred over the network **in an interlaced mode**, useful for low bit rate or packet networks. The **compressed data is divided into four groups**: **the first** contains **1/8** of the whole, **the second** a further **1/8**, **the third** a further **1/4**, and **the last** the remaining **1/2**



Tagged Image File Format (TIFF)

- 48-bit pixels, i.e., three 16-bits for each R, G and B are used
- Applicable for both images and digitized documents
 - code number 1: uncompressed formats
 - code number 2, 3 & 4: digitized documents as in FAX
 - code number 5: LZW-compressed formats

RUN:
repetitiveness
of data

Digitized Documents (FAX)

- ITU-T series for FAX documents: Group 3(G3) is for an analog PSTN: no error correcting function
 - G4 is a digitalized PSTN like ISDN: error correction
 - Usually 10:1 compression is attainable
- Two tables of codewords are given in advance
 - Termination-codes table: white or black runlengths from 0 to 63 pixels in step of 1 pixel
 - Make-up codes table: white or black runlengths that are multiple of 64 pixels

G3(T4) Code Tables

Termination-code Table

White run-length	Code- word	Black run-length	Code- word
0	00110101	0	0000110111
1	000111	1	010
.....			
11	01000	11	0000101
12	001000	12	0000111
.....			
51	01010100	51	000001010011
52	01010101	52	000000100100
.....			
62	00110011	62	000001100110
63	00110100	63	000001100111

Makeup-code Table

White run-length	Code- word	Black run-length	Code- word
64	11011	64	0000001111
128	10010	128	000011001000
.....			
640	01100111	640	0000001001010
704	011001100	704	0000001001011
.....			
1664	011000	1664	0000001100100
1728	010011011	1728	0000001100101
.....			
2560	000000011111	2560	000000011111
EOL	00000000001	EOL	00000000001

Digitized Documents(2): G3

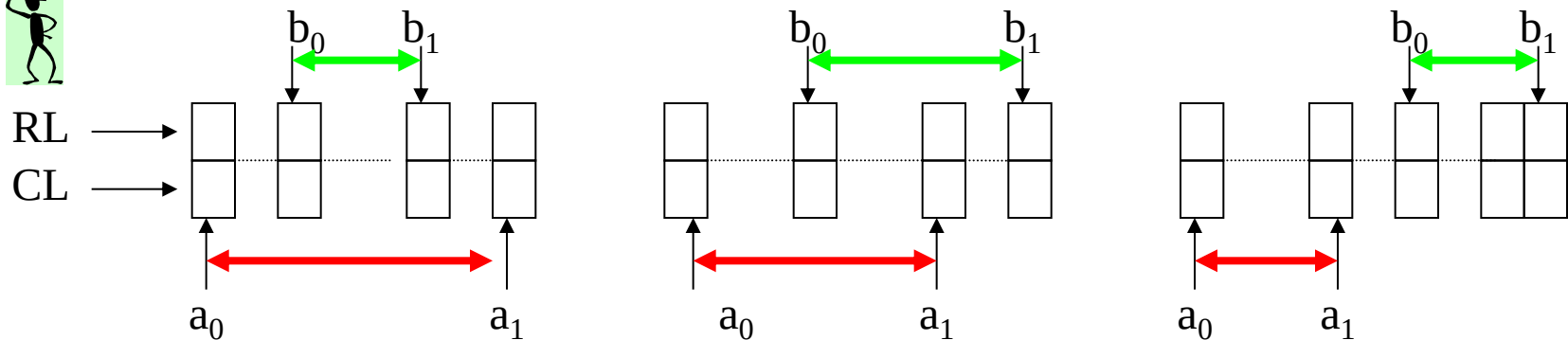
- The **overscanning** technique is used in G3(T4)
 - All lines start with a minimum of one white pixel
 - The receiver knows the first codeword always relates to white pixels and then alternates **between black and white**
 - Some coding examples: a runlength of 12 white pixels is coded directly as **001000** and a runlength of 12 black pixels is as **0000111**. Thus, a 140 black pixels is encoded **128+12 = 0000110010000000111**
 - Runlengths exceeding 2560 pixels are encoded using more than one **make-up code** plus one **termination code**

Digitized Documents(3): G4

- **MMR** (Modified-Modified READ) Coding, also known as **2-D Runlength Coding**
 - Optional in G3 but compulsory in G4 where, runlengths are identified by comparing **adjacent scan lines**.
 - **READ** stands for **Relative Element Address Designate**, and it is "**modified**" since it is a modified version of an earlier (modified) coding scheme
 - **Coding Idea**: **Most scanned lines differ from the previous lines by only a few pixels**
 - **Coding Line (CL)**: scanned line under encoding for compression
 - **Reference Line (RL)**: previously encoded line
 - **Assumption**: **the first RL** per page is always **all-white** line

Digitized Documents(4): G4

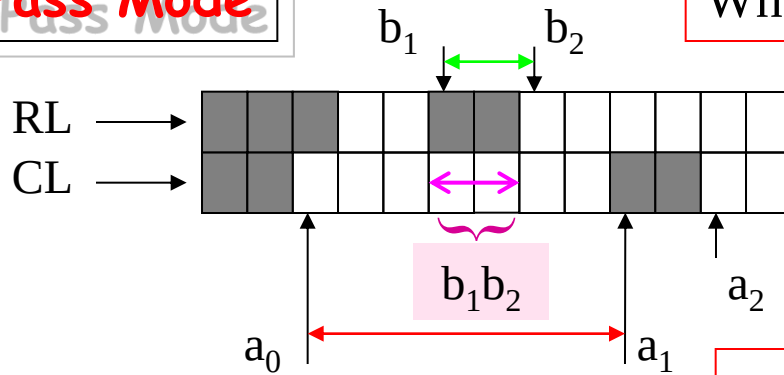
- MMR (Modified-Modified READ) Coding
 - Pass Mode
 - Vertical Mode
 - Horizontal Mode
- Notations
 - a_0 : 1st pixel of a new codeword, which is white (W) or black (B)
 - a_1 : 1st pixel to the right of a_0 with different color
 - a_2 : 1st pixel to the right of a_1 with different color
 - b_1 : 1st pixel on the RL to the right of a_0 with a different color
 - b_2 : 1st pixel on the RL to the right of b_1 with a different color



Digitized Documents(5): G4

Pass Mode

When b_2 lies to the left of a_1



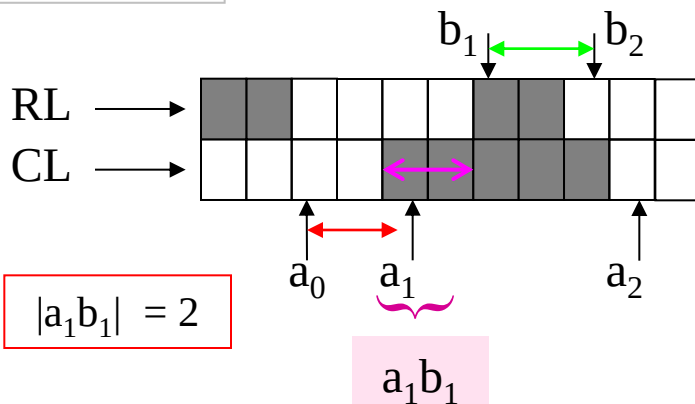
- 1) run-length b_1b_2 coded
- 2) new a_0 becomes old b_2

a_2 is the 1st pixel to the right of a_1 with different color

When a_1 is within 3 pixels to the left or right of b_1

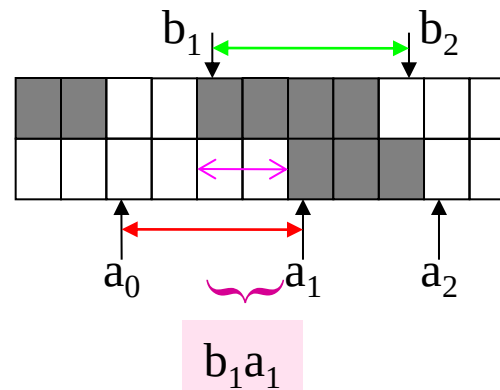
Vertical Mode

$$|a_1b_1| \leq \pm 3$$



$$|a_1b_1| = 2$$

- 1) run-length $a_1b_1(b_1a_1)$ coded
- 2) new a_0 becomes old b_2



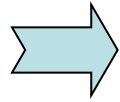
$|a_0a_1|$: no. of pixels from a_0 before (to) a_1 ?

$$|b_1a_1| = 2$$

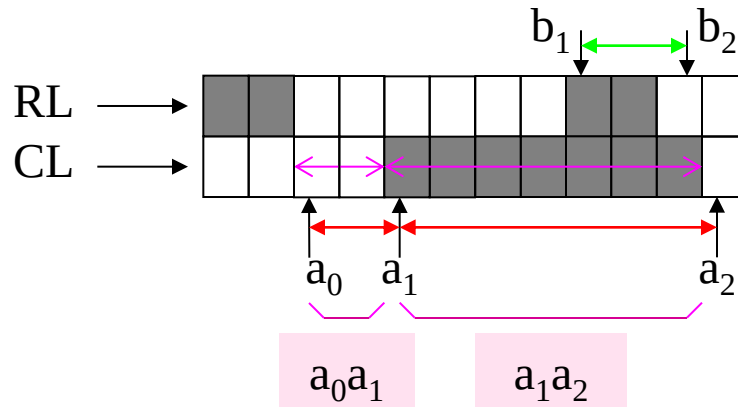
$$|a_1b_1| = -2$$

Digitized Documents(6): G4

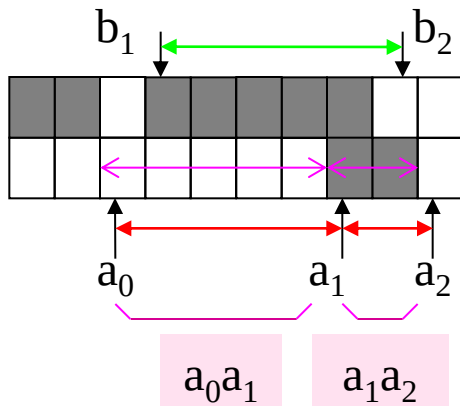
Horizontal Mode



$$|a_1 b_1| > \pm 3$$



$$|a_1 b_1| = 4$$



$$|b_1 a_1| = -4$$

$$|a_1 b_1| = 4$$

- 1) run-length $a_0 a_1$ coded white
- 2) run-length $a_1 a_2$ coded black
- 3) new a_0 becomes old b_2

Lossy Compression Algorithms: Transform Coding (1), DCT

- The rationale behind transform coding is that if $\mathbf{y}$ is the result of a linear transform $\mathbf{T}$ of the input vector $\mathbf{x}$ in such a way that the components of $\mathbf{y}$ are much less correlated, then $\mathbf{y}$ can be coded more efficiently than $\mathbf{x}$
- The transform $\mathbf{T}$ itself does not compress any data. The compression comes from the processing and quantization of the components of $\mathbf{y}$
- DCT (Discrete Cosine Transformation) is a tool to decorrelate the input signal in a data-independent manner.



Unlike 1D audio signal, a digital image $f(i,j)$ is not defined over the time domain. It is defined over a spatial domain, i.e., an image is a function of the 2D i and j (or x and y). For instance, The 2D DCT is used as one step in JPEG to yield a frequency response that is a function $F(u,v)$ in the spatial frequency domain indexed by two integers u and v .

Lossy Compression Algorithms: Transform Coding (5), DCT

Why DCT

- An electrical signal with constant magnitude is known as a DC (Direct Current), for instance, a battery that carries 1.5 or 9 volts DC. An electrical signal that changes its magnitude periodically at a certain frequency is known as an AC (Alternating Current) signal, say, 110 volts AC and 60 Hz (or 220 volts and 50 Hz)
- Most real signals are more complex, any signal can be expressed as sum of multiple signals that are sine or cosine waveforms at various amplitudes and frequencies
- If a cosine function is used, the process of determining the amplitude of the AC and DC components of the signal is called a Cosine Transform, and the integers indices make it a Discrete Cosine Transform.
- When $u=0$, Eq. (5) yields the DC coefficient; when $u=1$ or 2 or ... up to 7, it yields the first or second ... 7th AC coefficient.

Lossy Compression Algorithms: Transform Coding (6), DCT

Why DCT

- The DCT is to decompose the original signal into its DC and AC components while the IDCT is to reconstruct the signal
- Eq.(6) shows the IDCT. This uses a sum of the products of the DC or AC coefficients and the cosine functions to reconstruct (recompose) the function $f(i)$.
- Since the DCT and IDCT involves some loss, $f(i)$ is denoted by $\tilde{f}(i)$
- The DCT and IDCT use the same set of cosine functions known as basis functions
- The function $f(i,j)$ is in the time domain while the function $F(u,v)$ is in the space domain
- The coefficients $F(u,v)$ are known as the frequency response and form the frequency spectrum of $f(i)$

Lossy Compression Algorithms: Transform Coding (2), DCT

■ The definition of DCT

- Given a function $f(i,j)$ over two integer variables i and j , a piece of an image, the **2D DCT** transforms it into a new function $F(u,v)$, with integers u and v running over the same range as i and j .

$$F(u,v) = \frac{2C(u)C(v)}{\sqrt{MN}} \sum_{i=0}^{M-1} \sum_{j=0}^{N-1} \cos \frac{(2i+1)u\pi}{2M} \cos \frac{(2j+1)v\pi}{2N} f(i,j) \quad (1)$$

where $i, u = 0, 1, \dots, M-1$, and $j, v = 0, 1, \dots, N-1$. The $C(u)$ and $C(v)$ are determined by

$$C(u) = \begin{cases} \frac{\sqrt{2}}{2} & \text{if } u=0 \\ 1 & \text{otherwise} \end{cases} \quad (2)$$

Lossy Compression Algorithms: Transform Coding (3), DCT

- In the JPEG image compression standard a image block is defined to have dimension $M=N=8$, the **2D DCT** is as follows

- $$F(u,v) = \frac{C(u)C(v)}{4} \sum_{i=0}^7 \sum_{j=0}^7 \cos \frac{(2i+1)u\pi}{16} \cos \frac{(2j+1)v\pi}{16} f(i,j) \quad (3)$$

where $i, u = 0, 1, \dots, 7$, and $j, v = 0, 1, \dots, 7$. The $C(u)$ and $C(v)$ are determined by

$$C(u) = \begin{cases} \frac{\sqrt{2}}{2} & \text{if } u=0 \\ 1 & \text{otherwise} \end{cases} \quad (2)$$

Lossy Compression Algorithms: Transform Coding (4), DCT

- 2D IDCT (Inverse DCT)

$$\tilde{f}(i,j) = \sum_{u=0}^7 \sum_{v=0}^7 \frac{C(u)C(v)}{4} \cos \frac{(2i+1)u\pi}{16} \cos \frac{(2j+1)v\pi}{16} F(u,v) \quad (4)$$

where $i, j, u, v = 0, 1, \dots, 7$

- 1D DCT

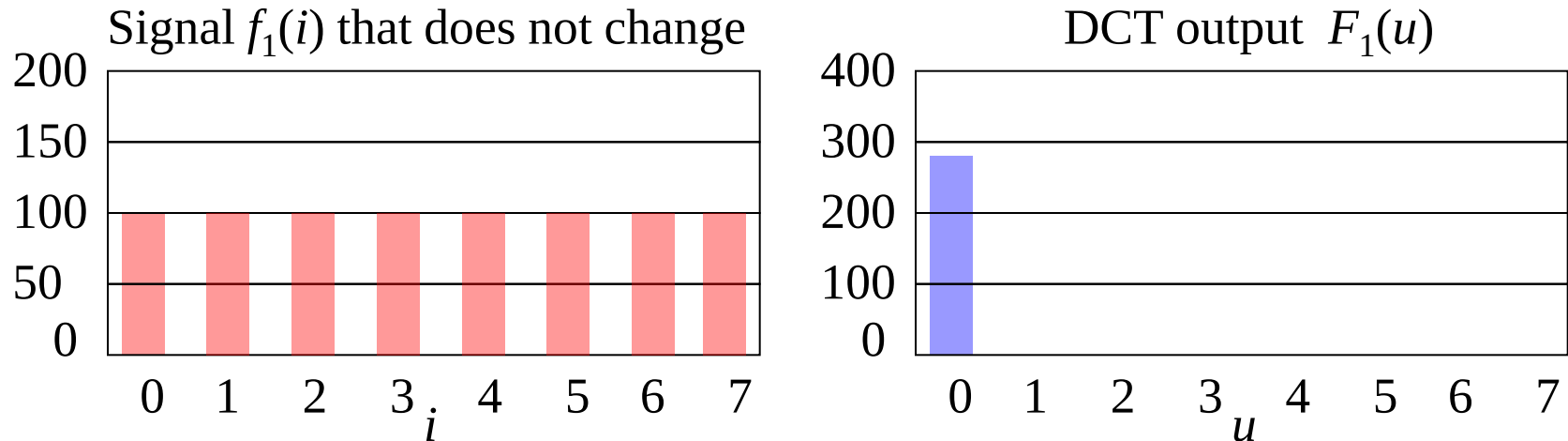
$$F(u) = \frac{C(u)}{2} \sum_{i=0}^7 \cos \frac{(2i+1)u\pi}{16} f(i) \quad (5)$$

- 1D IDCT

$$\tilde{f}(i) = \sum_{u=0}^7 \frac{C(u)}{2} \cos \frac{(2i+1)u\pi}{16} F(u) \quad (6)$$

Lossy Compression Algorithms: Transform Coding (7), DCT

Some Examples



The left figure shows a DC signal with a magnitude of 100, i.e., $f_1(i)=100$. When $u=0$, regardless of i , all the cosine terms in Eq.(5) become $\cos 0$, which equal 1. Taking into account that $C(0)=\sqrt{2}/2$, $F_1(0)$ is given by

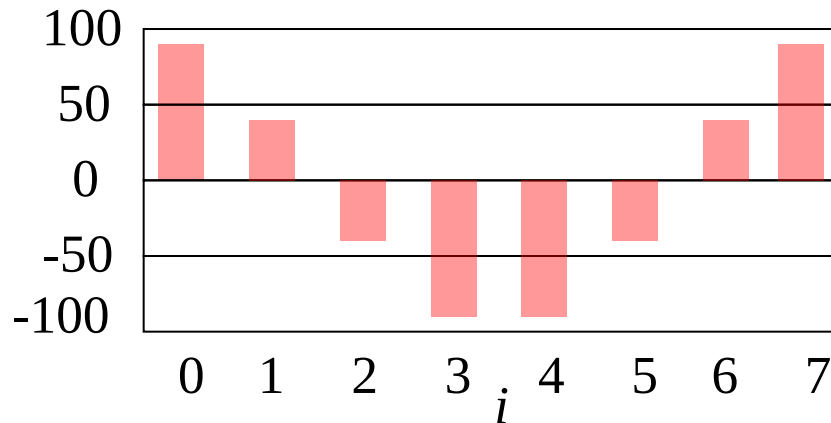
$$F_1(0) = \sqrt{2}/(2 \cdot 2) (1 \cdot 100 + 1 \cdot 100 + 1 \cdot 100 + 1 \cdot 100 + 1 \cdot 100 + 1 \cdot 100 + 1 \cdot 100) \approx 283$$

Similarly, it can be shown that $F_1(1)=F_1(2)=F_1(3)=\dots F_1(7)=0$

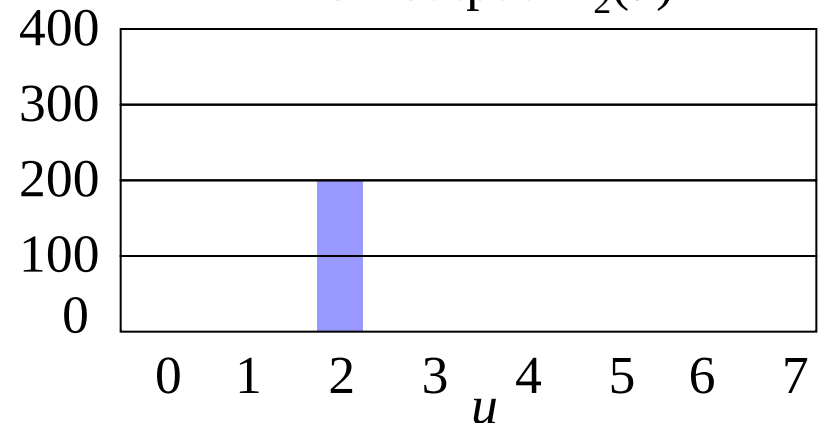
Lossy Compression Algorithms: Transform Coding (8), DCT

Some Examples

A changing signal $f_2(i)$ that has
an AC component



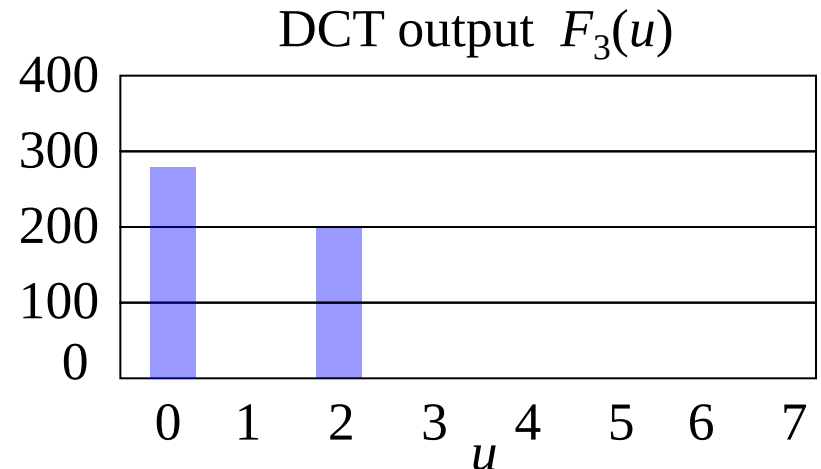
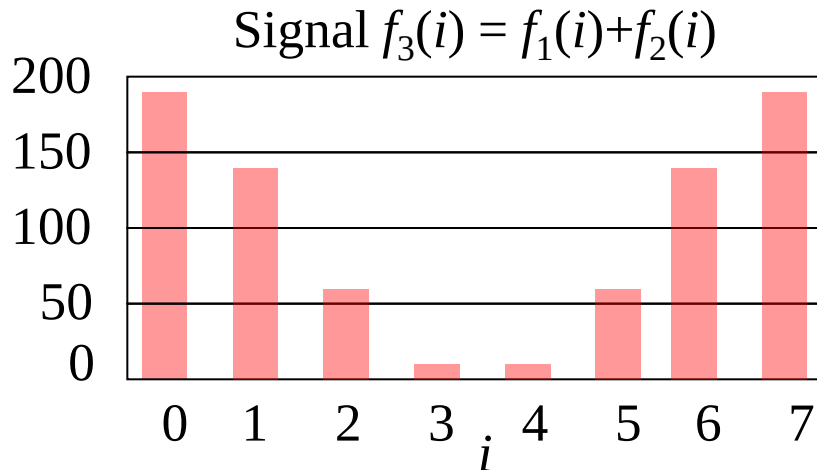
DCT output $F_2(u)$



The left figure shows an AC signal with a magnitude of 100, i.e., $f_1(i)=100$. It can be easily shown that $F_1(1)= F_1(3)= \dots F_1(7)= 0$ but $F_1(2)=200$.

Lossy Compression Algorithms: Transform Coding (9), DCT

Some Examples



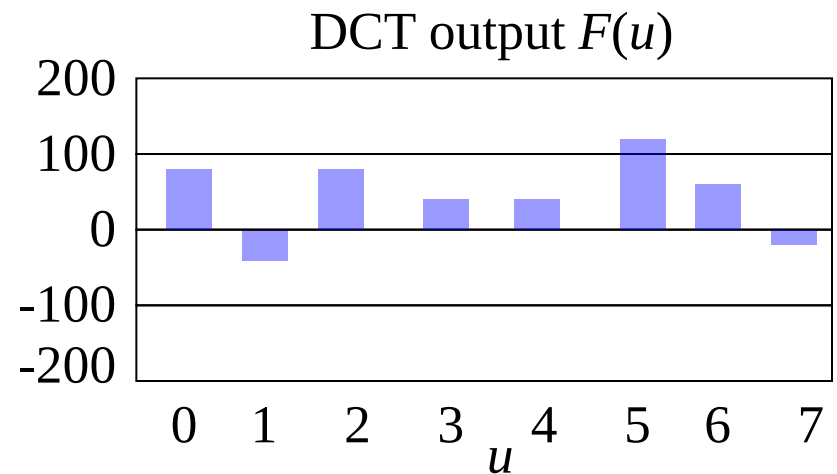
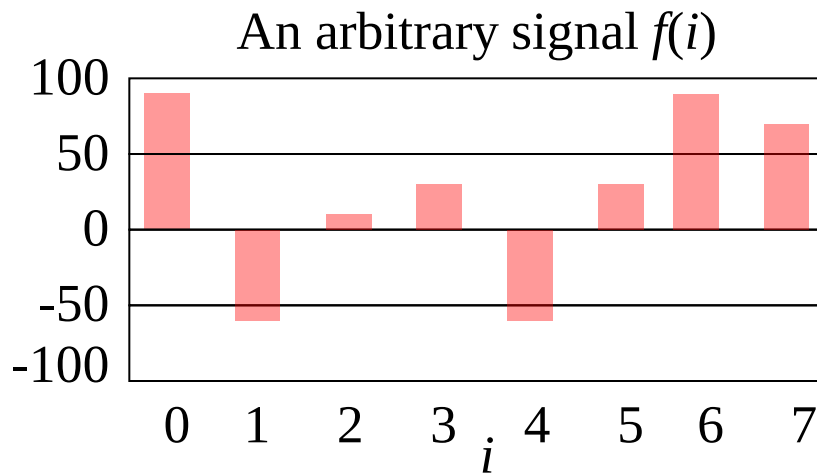
The input signal to the DCT is now the sum of the previous two signals, $f_3(i) = f_1(i) + f_2(i)$.

The output $F(u)$ values are $F_3(0) = 238$, $F_3(2) = 200$, and $F_3(1) = F_3(3) = F_3(4) = \dots = F_3(7) = 0$.

Again we discover that $F_3(u) = F_1(u) + F_2(u)$.

Lossy Compression Algorithms: Transform Coding (10), DCT

Some Examples



$f(i)(i=0,1,\dots,7)$: 85 -65 15 30 -56 35 90 60
 $F(u)(u=0,1,\dots,7)$: 69 -49 74 11 16 117 44 -5

Lossy Compression Algorithms: Transform Coding (11), DCT

Characteristic of the DCT

- The DCT produces the frequency spectrum $F(u)$ corresponding to the spatial signal $f(i)$
 - The 0th DCT coefficient $F(0)$ is the DC component of $f(i)$. Up to a constant factor $((1/2)(\sqrt{2}/2)(8)=2\sqrt{2}$ in the 1D DCT and $(1/4)(\sqrt{2}/2)(\sqrt{2}/2)(64)=8$ in the 2D DCT), $F(0)$ equals the average magnitude of the signal
 - The other seven DCT coefficients reflect the various changing (i.e., AC) components of the signal $f(i)$ at different frequencies.
- The cosine basis functions, say eight 1D DCT or IDCT functions for $u=0,\dots,7$, are orthogonal so as to have the least redundancy amongst them for a better decomposition.

Lossy Compression Algorithms: Transform Coding (12), Wavelet-Based Coding

- Another method decomposing the input signal into its constituents is **the wavelet transform**. It seeks to represent a signal with good resolution in both **time** and **frequency domain**, by using a set of basic functions called **wavelets**.
- The approach provides us a **multiresolution analysis**: Mentally stacking the **full-size** image, the **quarter-size** image, the **sixteen-size** image, and so on, creates a **pyramid**.

Lossy Compression Algorithms: Transform Coding (13), Wavelet-Based Coding

Some Examples

- Suppose we are give the input signal sequence $\{x_{n,i}\} = \{10, 13, 25, 26, 29, 21, 7, 15\}$ where, $i \in [0,7]$ indexes “pixels”, and n stands for the level of a pyramid we are on, in this case, at the top, $n=3$.
- Consider the transformation that replaces the original sequence with its pairwise average $x_{n-1,i}$ and difference $d_{n-1,i}$ defined as follows:

$$x_{n-1,i} = \frac{x_{n,2i} + x_{n,2i+1}}{2}$$

$$d_{n-1,i} = \frac{x_{n,2i} - x_{n,2i+1}}{2}$$

Lossy Compression Algorithms: Transform Coding (14), Wavelet-Based Coding

Some Examples

- $\{x_{n-1,i}, d_{n-1,i}\} = \{11.5, 25.5, 25, 11, -1.5, -0.5, 4, -4\}, i=0,1, \dots, 7.$
- The original sequence can be reconstructed from the transformed sequence using the relations

$$\begin{aligned} x_{n,2i} &= x_{n-1,i} + d_{n-1,i} \\ d_{n,2i+1} &= x_{n-1,i} - x_{n-1,i+1} \end{aligned}$$



$$x_{n-2,i} = [x_{n-1,2i} + x_{n-1,2i+1}]/2 = [11.5 + 25.5]/2 = 18.5$$

- $\{x_{n-2,i}, d_{n-2,i}, d_{n-1,i}\} = \{18.5, 18, -7, 7, -1.5, -0.5, 4, -4\}$
- $\{x_{n-3,i}, d_{n-3,i}, d_{n-2,i}, d_{n-1,i}\} = \{18.25, 0.25, -7, 7, -1.5, -0.5, 4, -4\}$

Average of elements in the original sequence

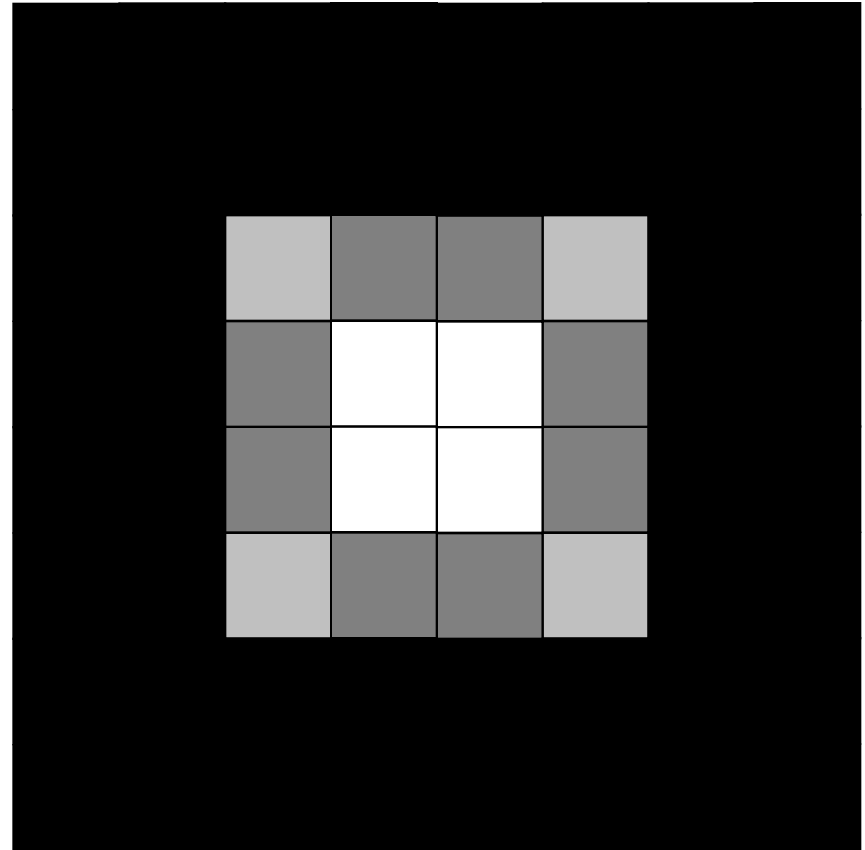
$$\{x_{n,i}\} = \{10, 13, 25, 26, 29, 21, 7, 15\}$$

Lossy Compression Algorithms: Transform Coding (15), Wavelet-Based Coding

Some Examples

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	63	127	127	63	0	0
0	0	127	255	255	127	0	0
0	0	127	255	255	127	0	0
0	0	63	127	127	63	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Pixel Value



Corresponding 8×8 image

Lossy Compression Algorithms: Transform Coding (16), Wavelet-Based Coding

Some Examples

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	95	95	0	0	-32	32	0
0	191	191	0	0	-64	64	0
0	191	191	0	0	-64	64	0
0	95	95	0	0	-32	32	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Intermediate Output of 2D
Haar Wavelet Transform

0	0	0	0	0	0	0	0
0	143	143	0	0	-48	48	0
0	143	143	0	0	-48	48	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	-48	-48	0	0	16	-16	0
0	48	48	0	0	-16	16	0
0	0	0	0	0	0	0	0

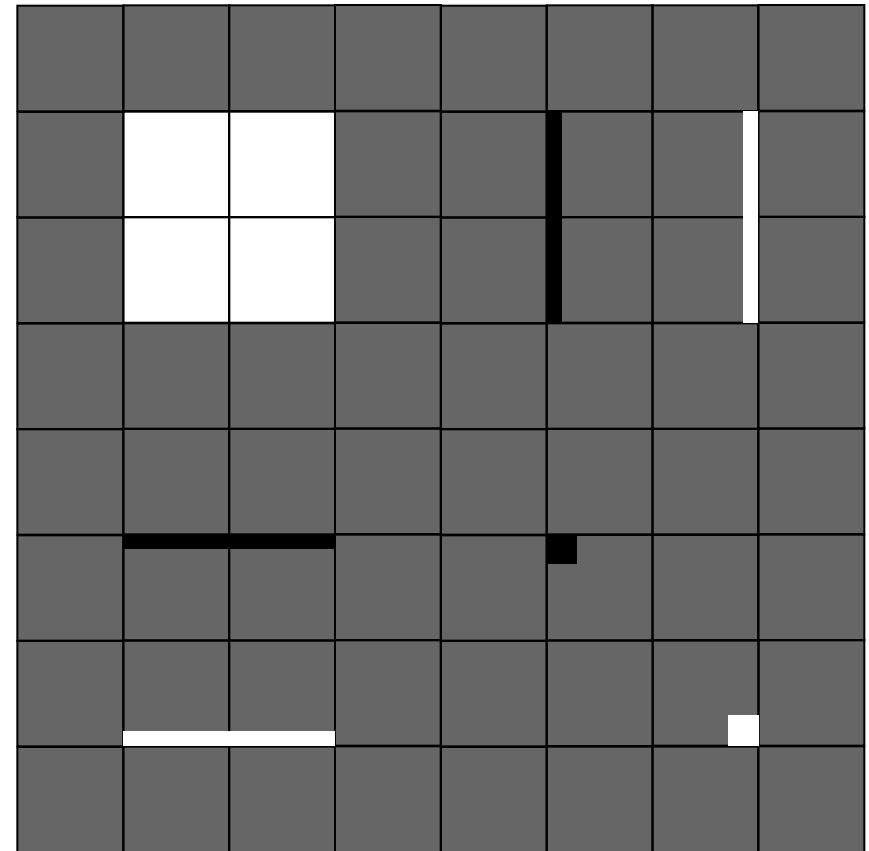
Output of the 1st Level of 2D
Haar Wavelet Transform

Lossy Compression Algorithms: Transform Coding (17), Wavelet-Based Coding

Some Examples

0	0	0	0	0	0	0	0
0	143	143	0	0	-48	48	0
0	143	143	0	0	-48	48	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	-48	-48	0	0	16	-16	0
0	48	48	0	0	-16	16	0
0	0	0	0	0	0	0	0

Output of the 1st Level of 2D
Haar Wavelet Transform



Corresponding Image

Digitized Pictures (Still Image): JPEG



1. Unlike 1D audio signal, a digital image $f(i,j)$ is not defined over the **time domain**. It is defined over a **spatial domain**, i.e., an image is a function of the 2D i and j (or x and y). For instance, the 2D DCT is used as one step in JPEG to yield a **frequency response** that is a function $F(u,v)$ in the **spatial frequency domain indexed by two integers u and v** .
2. Spatial frequency indicates how many times pixel values change across an image block. In the DCT this notion means **how much the image contents change in relation to the number cycles of a cosine wave per block**

Digitized Pictures (Still Image): JPEG

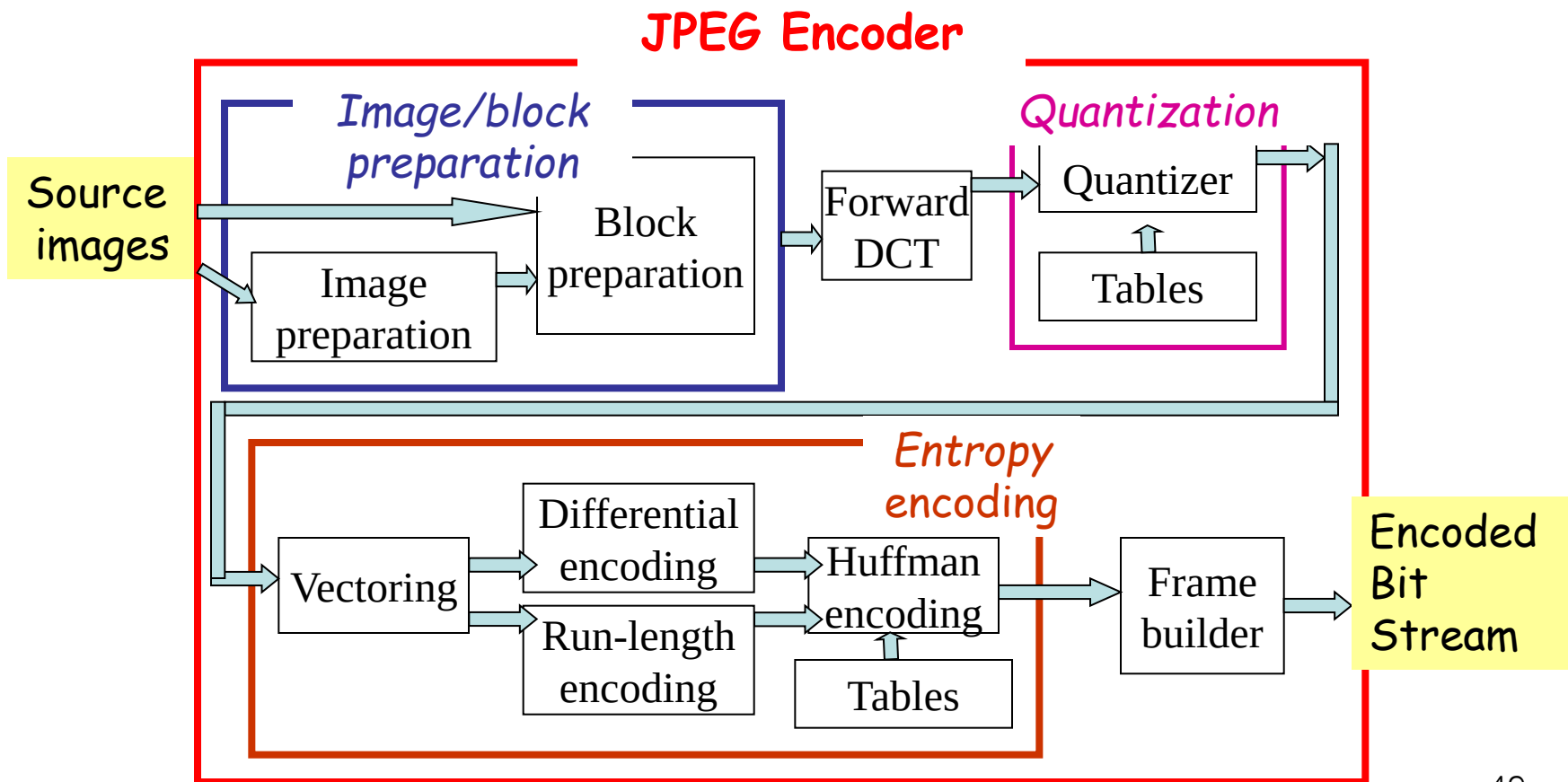


Effectiveness of the DCT transform coding in JPEG relies on three observations as follows.

1. Useful image *contents change relatively slowly* across the image
2. Psychophysical experiments suggest that humans are much less likely to notice the loss of very high-spatial-frequency components than lower-frequency components
 - JPEG's approach to the use of DCT is basically to *reduce high-frequency contents* and then efficiently code the result
 - *Spatial redundancy* means how much of the information in an image is repeated: *if a pixel is red, then its neighbor is likely red also*. As frequency gets higher, it becomes less important to represent the DCT coefficient accurately.
3. Visual accuracy in distinguishing closely spaced lines *is much greater for gray* (“black-white”) than for color.

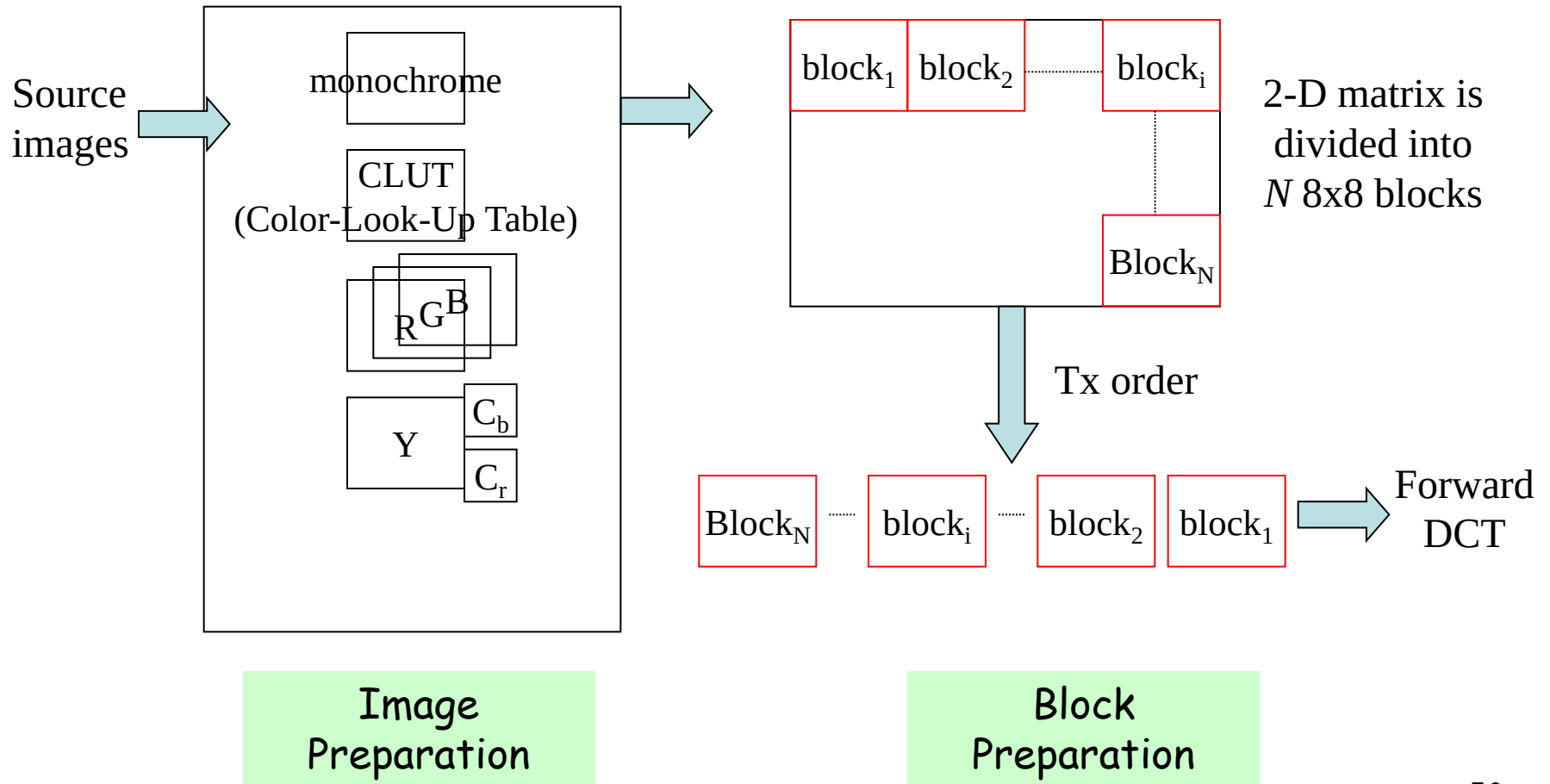
Digitized Pictures (Still Image): JPEG

- **JPEG**: *Joint Photographic Experts Group*
 - "Lossy Sequential Mode", also known as "Baseline Mode"
 - IS 10918 by ISO (in cooperation with ITU & IEC)



Digitized Pictures: JPEG(2)

■ Image/Block Preparation

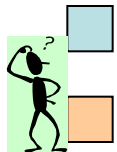
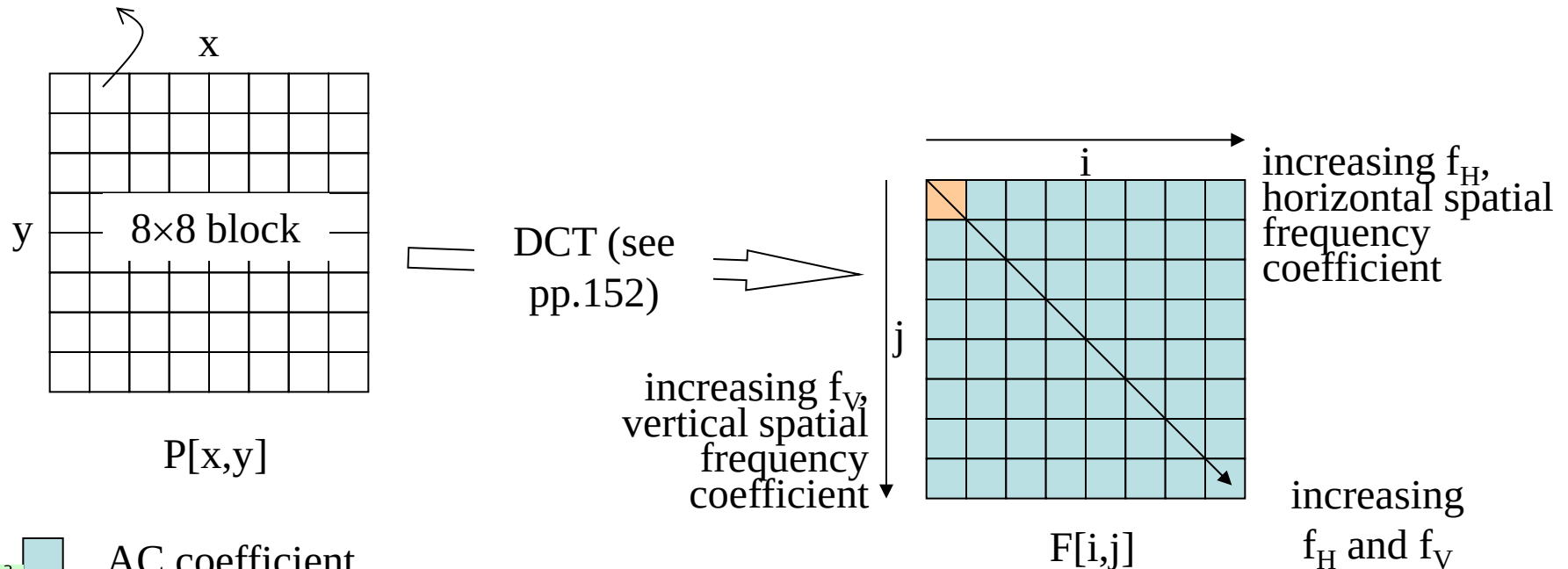


Digitized Pictures: JPEG(3)

- DCT (Discrete Cosine Transformation)

R/G/B or Y : [0, 255] levels

C_b/C_r : [-128, 127] levels



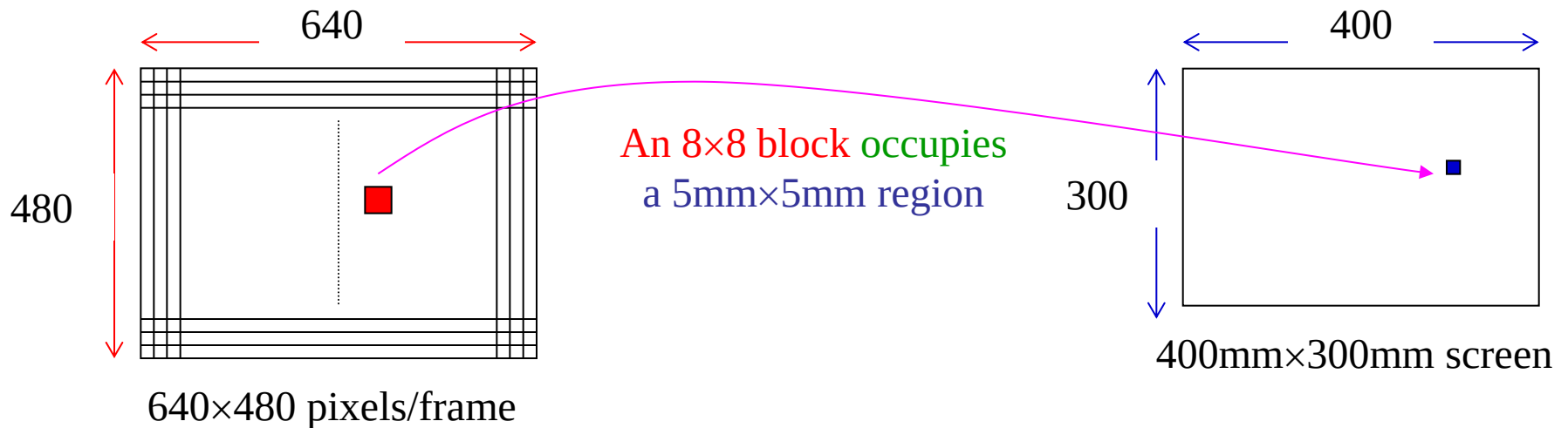
AC coefficient

DC coefficient: *mean of all 64 values averaging*
color/luminance/chrominance associated with an 8x8 block

Digitized Pictures: JPEG

- DCT (Discrete Cosine Transformation) Example

Consider a typical image frame comprising 640×480 pixels. Assuming a block of 8×8 pixels, the image will comprise 80×60 (4800) blocks each of which, for a screen width of, say, 16 inches (400mm), will occupy a square of only 0.2×0.2 inches (5×5 mm).



Those regions of a picture frame that contain a single (or similar) color (s) will generate a set of transformed blocks of all of which will have the same (or very similar) DC coefficient (s) and only a few (or little bit) different AC coefficient (s). The blocks of quite different AC (s) and DC (s) will generate very different colors.

Digitized Pictures: JPEG(4)

■ Quantization

- The human eyes respond primarily to the **DC coefficient** and the **lower spatial coefficient**. Hence, a **higher spatial frequency coefficient** which is below a certain threshold, that the eyes will not detect it, **is dropped** (**quantizing error** inevitable)
- Instead of comparing each coefficient with the coefficient threshold, **a division operation with quantization tables** is used for the reduction of the size of the DC & AC coefficients

120	60	40	30	4	3	0	0
70	48	32	3	4	1	0	0
50	36	4	4	2	0	0	0
40	4	5	1	1	0	0	0
5	4	0	0	0	0	0	0
3	2	0	0	0	0	0	0
1	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0

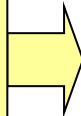
DCT Coefficients



10	10	15	20	25	30	35	40
10	15	20	25	30	35	40	50
15	20	25	30	35	40	50	60
20	25	30	35	40	50	60	70
25	30	35	40	50	60	70	80
30	35	40	50	60	70	80	90
35	40	50	60	70	80	90	100
40	50	60	70	80	90	100	110

Quantization Table

q
u
a
n
t
i
z
e
r



12	6	3	2	0	0	0	0
7	3	2	0	0	0	0	0
3	2	0	0	0	0	0	0
2	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Quantized Coefficients

Two default tables: one for luminance coefficients & the other for two chrominance coefficients



rounded to the nearest integer

DC coefficient is the largest

Most are zero (high spatial coefficients)

Digitized Pictures: JPEG(5)

Example 3.4

- Consider a quantization threshold value of 16. Derive the resulting quantization error for each of the following DCT coefficients:

127, 72, 64, 56, -56, -64, -72, -128

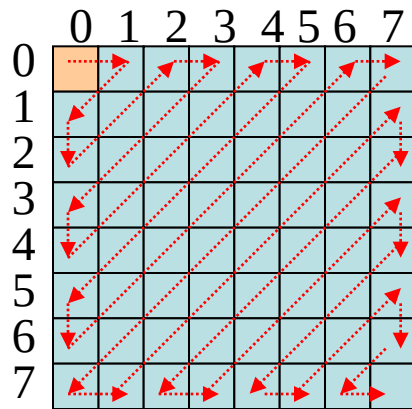
Coefficient	Quantized Value	Rounded Value	Dequantized Value	Error
127	$127/16 = 7.9375$	8	$8 \times 16 = 128$	+1
72	4.5	5	80	+8
64	4	4	64	0
56	3.5	4	64	+8
-56	-3.5	-4(-3)	-64(-48)	-8(+8)
-64	-4	-4	-64	0
-72	-4.5	-5(-4)	-80(-64)	-8(+8)
-128	-8	-8	-128	0

Max error/threshold = $8/16 \Rightarrow$ max error is **within 50%** of the threshold

Digitized Pictures: JPEG(6)

■ Entropy Encoding: Vectoring

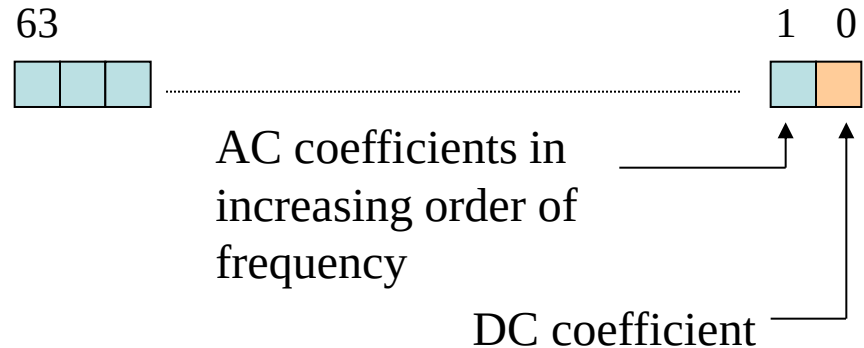
- Entropy Encoding Step : vectoring → differential encoding (DC coefficients) → run-length encoding (AC coefficients) → Huffman encoding



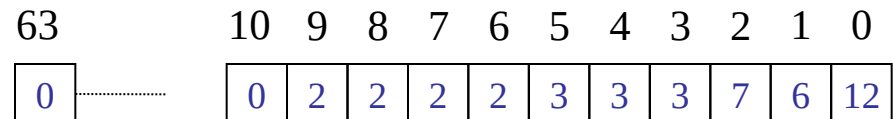
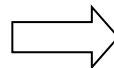
➡

**Zig-zag
Scanning**

Linearized vector(1-D vectorization)



12	6	3	2	0	0	0	0
7	3	2	0	0	0	0	0
3	2	0	0	0	0	0	0
2	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0



Digitized Pictures: JPEG(7)

- **Entropy Encoding: Differential Encoding for a DC coefficient**
 - A **DC coefficient** is a measure of the average color, luminance, and chrominance associated with the corresponding 8×8 block of pixels
 - Say, the sequence of DC coefficients 12, 13, 11, 11, 10, ... will generate the corresponding difference values 12, 1, -2, 0, 1, ($d_i = DC_i - C_{i+1}$, $i=1,2,\dots$)
 - Then, **only the difference** in magnitude of the DC coefficient in a quantized block relative to the value in the preceding block is encoded in the form of **<SSS, value>** where **SSS** indicates the number of bits needed to encode the value.

Difference value	No of Coefficients	SSS	Encoded value
0	1	0	0
-1, 1	2	1	-1=0, 1=1
-3, -2, 2, 3	4	2	-3=00, -2=01 2=10, 3=11
-7...-4, 4...7	8	3	-7=000...-4=011 4=100...7=111

1's complement of each other

Digitized Pictures: JPEG(8)

- Entropy Encoding: Differential Encoding for a DC coefficient

Example 3.5

Assume the sequence of DC coefficients is 12, 13, 11, 11, 10.
Find the difference values and the encoding values

The difference values are 12, 1, -2, 0, -1 and their encoded values are as follows

Value	SSS	Encoded value
12	4	1100
1	1	1
-2	2	01
0	0	
-1	1	0

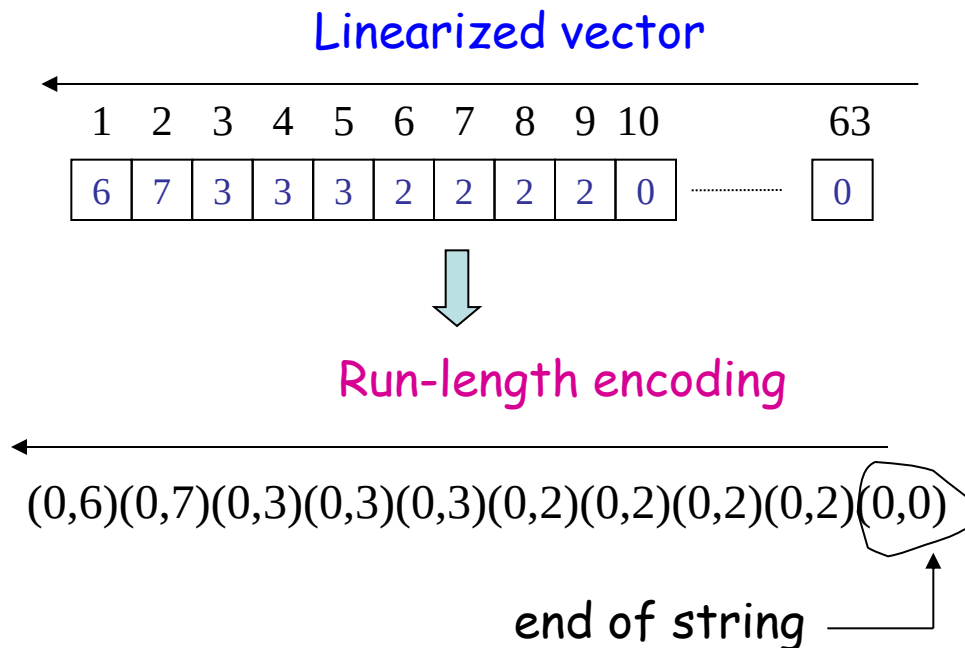
Diagram illustrating the encoding of difference values:

- For the difference value 1, the encoded value is 1.
- For the difference value -2, the encoded value is 01. This is shown with a diagram where 01 is in a circle, and an arrow labeled "1's complement" points to 10(2) in a circle.
- For the difference value 0, the encoded value is 0.
- For the difference value -1, the encoded value is 0. This is shown with a diagram where 0 is in a circle, and an arrow labeled "1's complement" points to 1(1) in a circle.

The final encoded code is 1100 1 01 0. This is a **DPCM** (differential PCM-Pulse Code Modulation) coding (also, see example 3.7 for detail)

Digitized Pictures: JPEG(9)

- **Entropy Encoding: Run-length Encoding for AC Coefficients**
 - The 63 remaining 8×8 blocks of pixels, AC coefficients, contain usually long strings of zeros within them
 - To exploit this feature each AC coefficient is encoded in form of a string of pairs of value (*skip*, *value*) where *skip* is the number of zeros in the run and *value* is the next non-zero coefficient



Digitized Pictures: JPEG(10)

■ Entropy Encoding: Run-length Encoding for AC Coefficients

Example 3.6

Derive the binary form of the following run-length encoded AC coefficients: (0,6)(0,7)(3,3)(0,-1)(0,0)

The sequence of AC coefficients:

6	7	0	0	0	3	-1
---	---	---	---	---	---	----

AC coefficients	Skip	SSS / Value
0,6	0	3 110
0,7	0	3 111
3,3	3	2 11
0,(-1)	0	1 0 ↔ 1(+1)
0,0	0	0

1's complement

Digitized Pictures: JPEG(11)

- Entropy Encoding: Huffman Encoding
 - The DC coefficients encoding

Default Huffman codeword for DC coefficients (Fig.3-19)

SSS Huffman encoded SSS

0	010
1	011
2	100
3	00
4	101
5	110
6	1110
7	11110
⋮	
11	11111110

Example 3.7

Determine the Huffman-encoded version of the following difference values which relates to the encoded DC coefficients from consecutive DCT blocks: 12, 1, -2, 0, -1

Difference values

SSS

Huffman-encoded SSS

Encoded value

Encoded bitstream sent

12	4	101	1100	1011110
1	1	011	1	0111
-2	2	100	01	10001
0	0	010		010
-1	1	011	0	0110

Digitized Pictures: JPEG(12)

- **Entropy Encoding: Huffman Encoding**
 - The **AC coefficient** encoding: skip & value fields are treated as a single symbol, and this is encoded using either the default Huffman code table or some table sent with the encoded bitstream

Example 3.8

Derive the composite binary symbols for the following set of runlength encoded AC coefficients:
(0,6)(0,7)(3,3)(0,-1)(0,0)

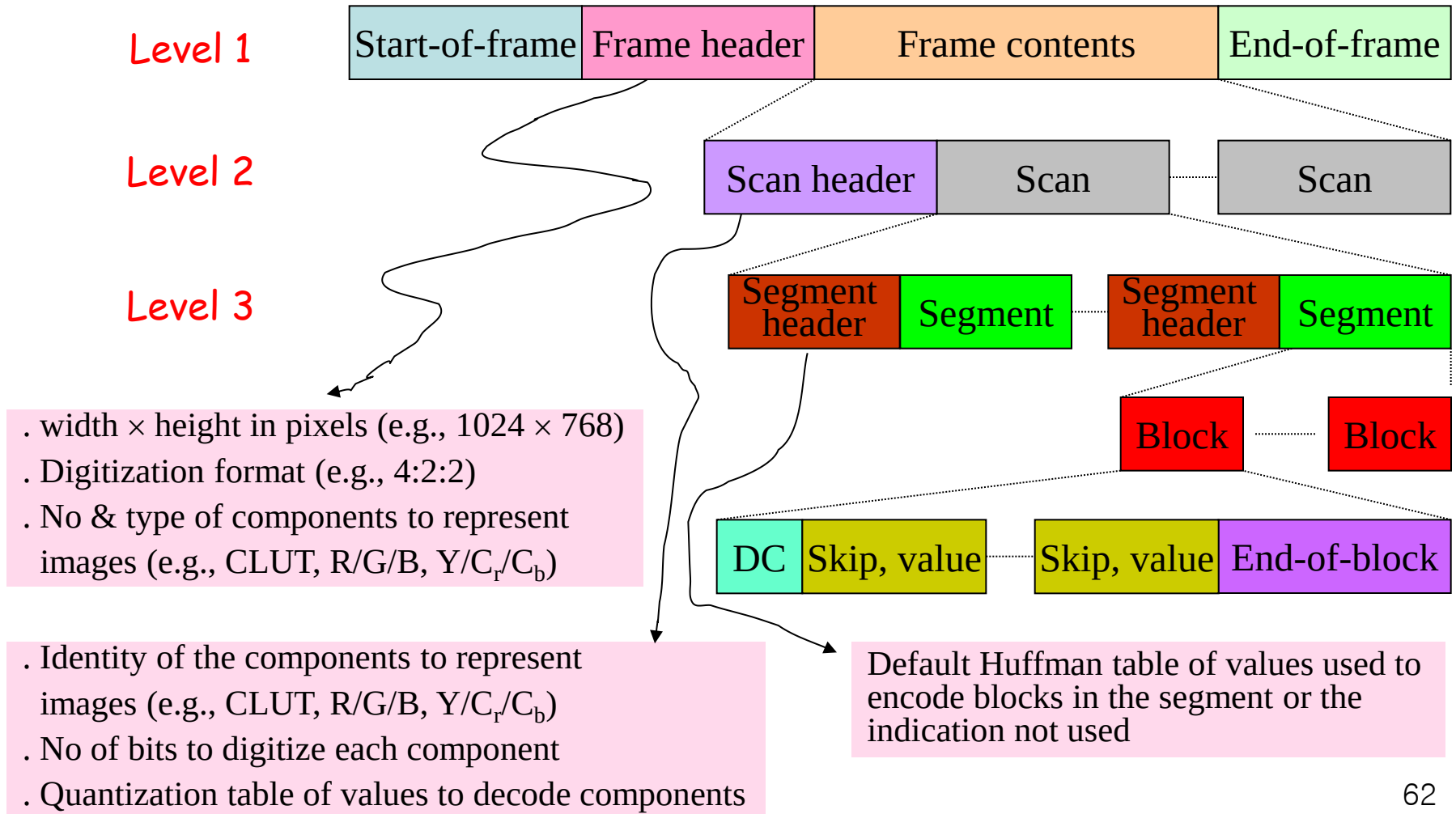
Default Huffman codeword for AC coefficients (Table 3.7)

Skip/SSS	Huffman encoded SSS	AC coefficients	skip	SSS	Huffman codewords	Runlength value
⋮	⋮	(0,6)	0	3	100	6=110
0/3	100	(0,7)	0	3	100	7=111
3/2	111110111	(3,3)	3	2	111110111	3=11
0/1	00	(0,-1)	0	1	00	-1=0
0/0	1010(=EOB)	(0,0)	0	0	1010	
⋮	⋮					

Bitstream sent: 100110100111111110111110001010⁶¹

Digitized Pictures: JPEG(13)

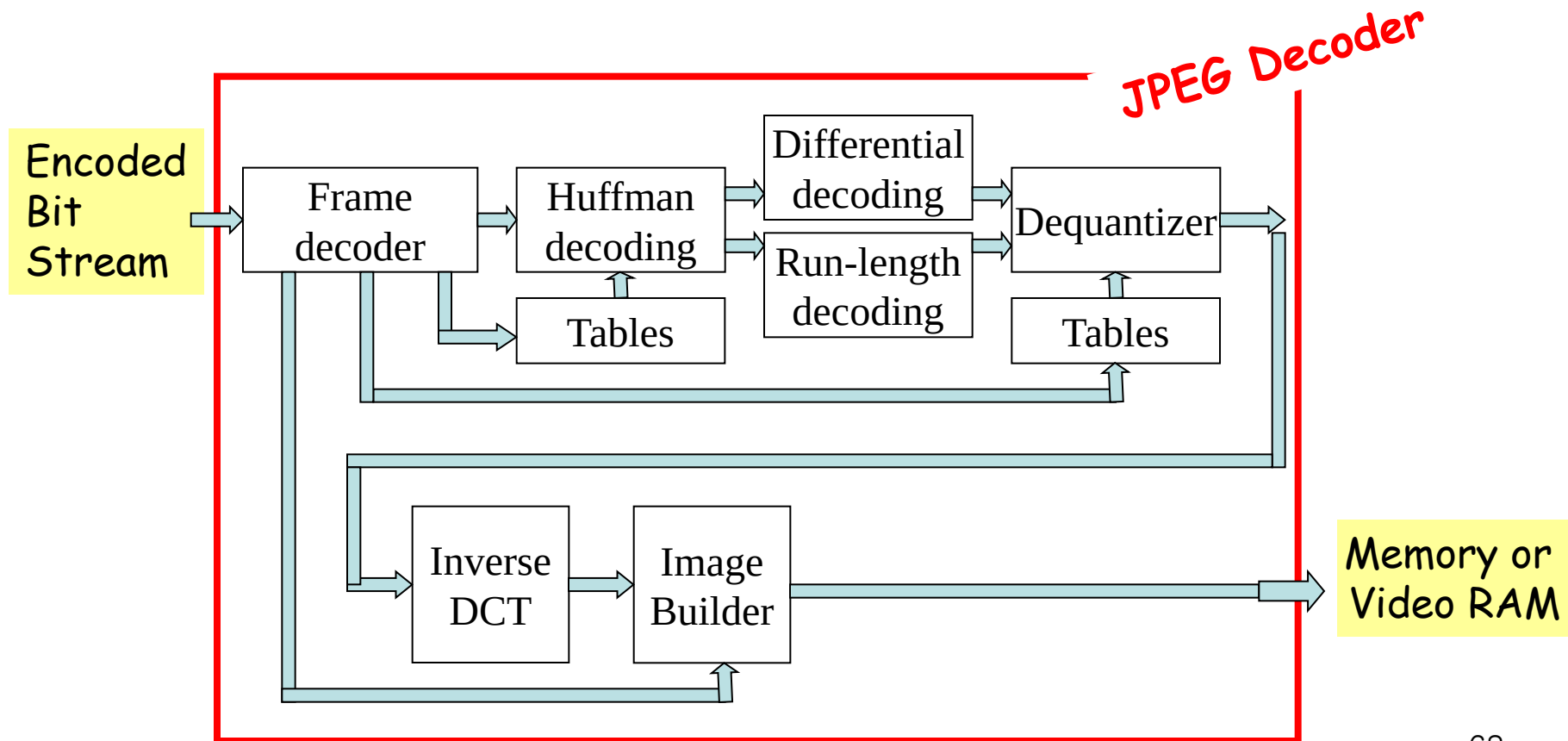
■ Frame Building: Hierarchical structure



Digitized Pictures: JPEG(14)

- **JPEG:** Decoding

- **Progressive mode:** DC and low-frequency coefficients first, then high-frequency coefficients (in *zig-zag scan mode* as Fig. 3-18)
- **Hierarchical mode:** total image with *low resolution* say, 320×240 first, *then at a higher resolution* say, 640×480



Digitized Pictures: JPEG(15)

- **JPEG Mode**

- **Sequential mode (Baseline mode)**

- **Progressive mode**

- **Spectral selection**

- Scan 1: Encode DC and first few AC components, e.g., AC1, AC2.

- Scan 2: Encode a few more AC components, e.g., AC3, AC4, AC5.

-

- Scan k : Encode the last few ACs, e.g., AC61, AC62, AC63.

- **Successive approximation**

- Scan 1: Encode the first few MSBs, e.g., Bits 7, 6, and 5.

- Scan 2: Encode a few more less-significant bits, e.g., Bit 3.

-

- Scan m : Encode the least significant bit (LSB), bit 0.

- **Hierarchical mode**: total image with *low resolution* say, 320×240 first, *then at a higher resolution* say, 640×480

Digitized Pictures: JPEG(16)

- JPEG2000 Standard
 - Low-bit rate compression
 - Transmission in nosy environments
 - Progressive Transmission
 - Region-of-interest coding
 - Computer-generated imagery
 - Supporting 256 channels
 - Wavelet-based transformation